Code Explanation

Test Data Processing Code Explanation

This code is a unit test suite for a data processing pipeline implemented using PySpark. The test suite covers the functionality of the `DataProcessor` class, which is responsible for cleaning and processing customer and order data.

Overview of the Code

The code is a test class that inherits from `unittest.TestCase`. It tests the `DataProcessor` class by creating test data, cleaning and processing it, and verifying the results. The test class uses PySpark to create DataFrames and test the data processing pipeline.

Step 1: Importing Necessary Modules and Setting Up the Test Environment
The code starts by importing the necessary modules, including `unittest`, `pyspark.sql`, and `datetime`. It also adds the `src` directory to the system path to enable imports from that directory.

```
import unittest
from unittest.mock import MagicMock, patch
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, I
ntegerType, DoubleType, DateType
import datetime
import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), '..'))
```

Step 2: Creating a Test Class and Setting Up the Spark Session
The test class `TestDataProcessor` is defined, and it includes a `setUpClass` method that creates a Spark session for the tests.

```
class TestDataProcessor(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.spark = SparkSession.builder
            .appName("TestDataProcessor")
            .master("local[1]")
            .getOrCreate()
```

Step 3: Creating Test Data and DataFrames
The `setUp` method creates test data for customers and orders, defines the schema for the DataFrames, and creates the DataFrames using the `createDataFrame` method.

```
def setUp(self):
    self.customer_data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C004", "Alice Brown", "alice@example.com", "West"),
```

```
        ("C005", "Null", "null@example.com", "North"),
        (None, "Invalid Customer", "invalid@example.com", "South"),
        ("C001", "John Doe", "john@example.com", "North")  # Duplica
te
    ]

    customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    self.customer_df = self.spark.createDataFrame(self.customer_data
, schema=customer_schema)

    # Similar code for order data
```

Step 4: Testing the `clean_data` Method
The `test_clean_data` method tests the `clean_data` method of the `DataProcessor` class by cleaning the customer and order DataFrames and verifying that the resulting DataFrames have the expected number of rows.

```
def test_clean_data(self):
    clean_customer_df = self.processor.clean_data(self.customer_df)
    self.assertEqual(clean_customer_df.count(), 4)

    clean_order_df = self.processor.clean_data(self.order_df)
    self.assertEqual(clean_order_df.count(), 6)
```

Step 5: Testing the `process_data` Method
The `test_process_data` method tests the `process_data` method of the `DataProcessor` class by mocking the `read_source_data` method and verifying that the subsequent methods are called correctly.

```
@patch('src.data_processing.DataProcessor.read_source_data')
def test_process_data(self, mock_read_source):
    mock_read_source.return_value = (self.customer_df, self.order_df
)

    self.processor.create_ordersummary_table = MagicMock()
    self.processor.create_customeraggregatespend_table = MagicMock()

    self.spark.sql = MagicMock()

    with patch.object(self.processor, 'load_scd_type2') as mock_load
_scd:
        with patch.object(self.processor, 'aggregate_customer_spend'
) as mock_aggregate:
            self.processor.process_data()

            mock_read_source.assert_called_once()
            mock_load_scd.assert_called_once()
```

```
        mock_aggregate.assert_called_once()
```